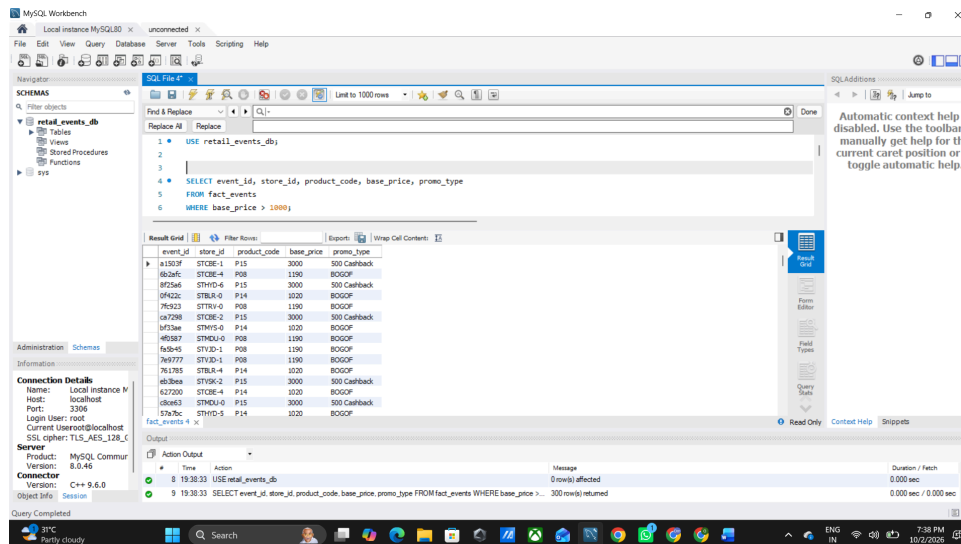


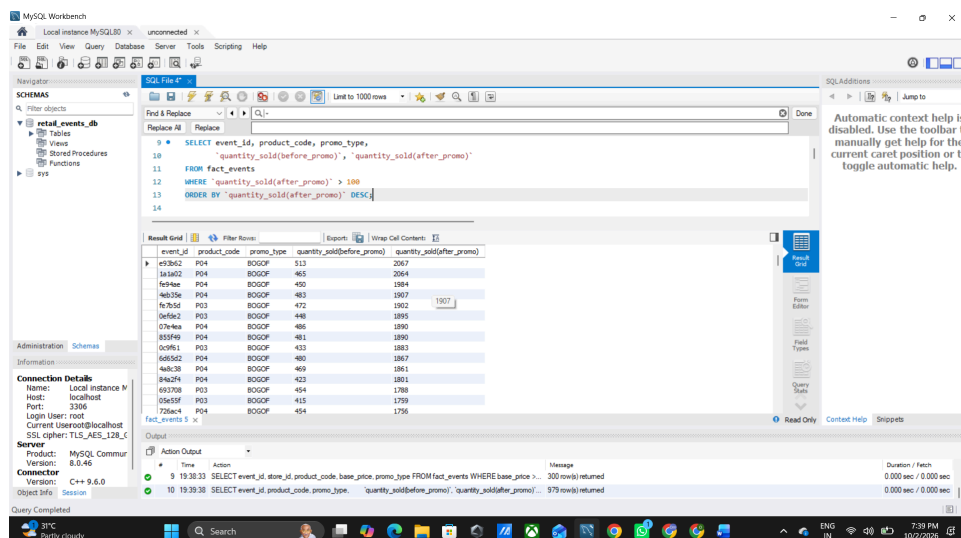
USE retail_events_db;

```
SELECT event_id, store_id, product_code, base_price, promo_type  
FROM fact_events  
WHERE base_price > 1000;
```



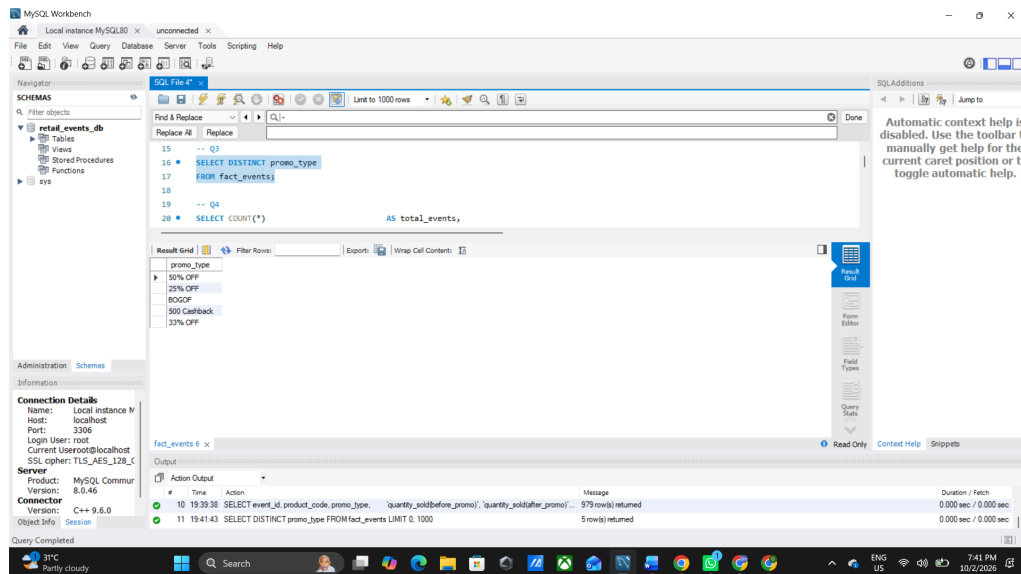
-- Q2

```
SELECT event_id, product_code, promo_type,  
       `quantity_sold(before_promo)`, `quantity_sold(after_promo)`  
FROM fact_events  
WHERE `quantity_sold(after_promo)` > 100  
ORDER BY `quantity_sold(after_promo)` DESC;
```



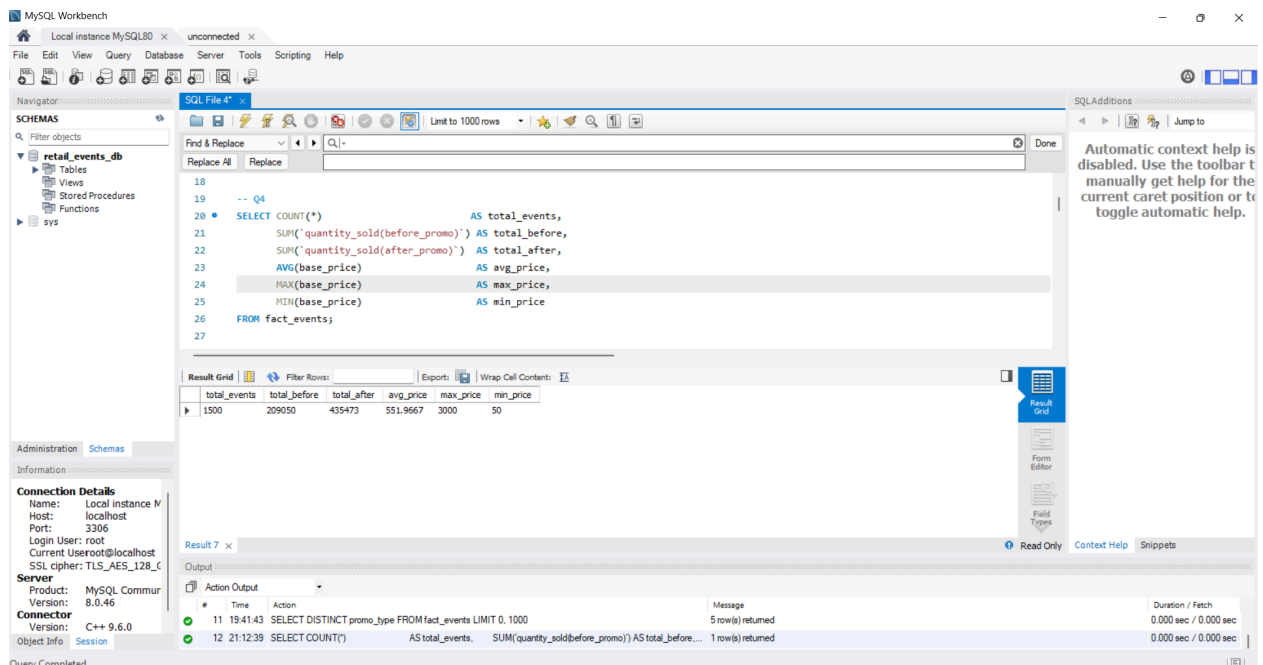
-- Q3

```
SELECT DISTINCT promo_type  
FROM fact_events;
```



-- Q4

```
SELECT COUNT(*) AS total_events,
SUM('quantity_sold(before_promo)') AS total_before,
SUM('quantity_sold(after_promo)') AS total_after,
AVG(base_price) AS avg_price,
MAX(base_price) AS max_price,
MIN(base_price) AS min_price
FROM fact_events;
```



-- Q5

```
SELECT promo_type,
COUNT(*) AS event_count,
SUM('quantity_sold(before_promo)') AS total_before,
```

```

SUM(`quantity_sold(after_promo)`) AS total_after
FROM fact_events
GROUP BY promo_type
ORDER BY total_after DESC;

```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

32 SELECT promo_type,
33        COUNT(*) AS event_count,
34        SUM(`quantity_sold(before_promo)`) AS total_before,
35        SUM(`quantity_sold(after_promo)`) AS total_after
36 FROM fact_events
37 GROUP BY promo_type
38 ORDER BY total_after DESC;

```

The Results grid shows the following data:

promo_type	event_count	total_before	total_after
BOGOF	500	58180	215253
33% OFF	200	63321	90576
500 Cashback	100	22299	63180
25% OFF	400	44007	36290
50% OFF	300	21243	28174

The Connection Details panel on the left shows the following information:

- Name: Local instance MySQL80
- Host: localhost
- Port: 3306
- Login User: root
- Current User: root@localhost
- SSL cipher: TLS_AES_128_GCM_SHA256
- Product: MySQL Community Server
- Version: 8.0.46
- Connector Version: C++ 9.6.0

```

-- Q6
SELECT promo_type,
SUM(`quantity_sold(before_promo)`) AS total_before,
SUM(`quantity_sold(after_promo)`) AS total_after,
SUM(`quantity_sold(after_promo)`) - SUM(`quantity_sold(before_promo)`) AS
quantity_change
FROM fact_events
GROUP BY promo_type
ORDER BY quantity_change DESC;

```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

43 SELECT promo_type,
44        SUM(`quantity_sold(before_promo)`) AS total_before,
45        SUM(`quantity_sold(after_promo)`) AS total_after,
46        SUM(`quantity_sold(after_promo)`) - SUM(`quantity_sold(before_promo)`) AS quantity_change
47 FROM fact_events
48 GROUP BY promo_type
49 ORDER BY quantity_change DESC;

```

The Results grid shows the following data:

promo_type	total_before	total_after	quantity_change
BOGOF	58180	215253	157073
500 Cashback	22299	63180	40881
33% OFF	63321	90576	27255
50% OFF	21243	28174	6931
25% OFF	44007	36290	-5717

The Connection Details panel on the left shows the following information:

- Name: Local instance MySQL80
- Host: localhost
- Port: 3306
- Login User: root
- Current User: root@localhost
- SSL cipher: TLS_AES_128_GCM_SHA256
- Product: MySQL Community Server
- Version: 8.0.46
- Connector Version: C++ 9.6.0

-- Q7

```
SELECT product_code, product_name, category,  
       SUM('quantity_sold(after_promo)') AS total_after  
FROM fact_events f  
JOIN dim_products p USING (product_code)  
GROUP BY product_code, product_name, category  
ORDER BY total_after DESC;
```

MySQL Workbench interface showing the execution of SQL query Q7. The query is displayed in the SQL Editor, and the results are shown in the Result Grid. The Result Grid displays columns: product_code, product_name, category, and total_after. The results are sorted by total_after in descending order. The bottom panel shows the Action Output with the query execution details.

product_code	product_name	category	total_after
P04	Atliq_Farm_Chalki_Atta (1KG)	Grocery & Staples	81290
P03	Atliq_Suflower_Oil (1L)	Grocery & Staples	74478
P15	Atliq_Home_Essential_8_Pr	Atliq_Home_Essential_8_Product_Comb	33220
P02	Atliq_Sonamasuri_Rice (10kg)	Grocery & Staples	33220
P01	Atliq_Masoor_Dal (1KG)	Grocery & Staples	37341
P13	Atliq_High_Glo_15W_LED_Bulb	Home Appliances	29928
P14	Atliq_waterproof_Immersion_Rod	Home Appliances	23685
P07	Atliq_Curtains	Home Care	16317
P08	Atliq_Double_Bedsheet_set	Home Care	15058
P12	Atliq_1mm_Cool_Raincoat_Rain (175GM)	Personal Care	10280

-- Q8

```
SELECT category,  
       COUNT(*) AS event_count,  
       SUM('quantity_sold(before_promo)') AS total_before,  
       SUM('quantity_sold(after_promo)') AS total_after,  
       SUM('quantity_sold(after_promo)') - SUM('quantity_sold(before_promo)') AS  
quantity_change  
FROM fact_events f  
JOIN dim_products p USING (product_code)  
GROUP BY category  
ORDER BY total_after DESC;
```

MySQL Workbench

Local instance MySQL80 x unconnected x

File Edit View Query Database Server Tools Scripting Help

Navigator

SQL File 4*

Find & Replace

Replace All Replace

```

63 SELECT category,
64     COUNT(*) AS event_count,
65     SUM('quantity_sold(before_promo)') AS total_before,
66     SUM('quantity_sold(after_promo)') AS total_after,
67     SUM('quantity_sold(after_promo)') - SUM('quantity_sold(before_promo)') AS quantity_change
68 FROM fact_events f
69 JOIN dim_products p USING (product_code)
70 GROUP BY category
71 ORDER BY total_after DESC;
72

```

Result Grid

category	event_count	total_before	total_after	quantity_change
Grocery & Staples	400	126970	246344	119374
Combo1	100	22299	63180	40881
Home Appliances	200	14713	53613	38900
Home Care	400	19764	49832	21068
Personal Care	400	25304	31504	6200

Connection Details

Name: Local instance M
Host: localhost
Port: 3306
Login User: root
Current User: root@localhost
SSL cipher: TLS_AES_128_GCM_SHA256

Server

Product: MySQL Commur
Version: 8.0.46

Connector

Version: C++ 9.6.0

Object Info

Query Completed

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

-- Q9

```

SELECT city,
       COUNT(*) AS event_count,
       SUM('quantity_sold(before_promo)') AS total_before,
       SUM('quantity_sold(after_promo)') AS total_after
FROM fact_events f
JOIN dim_stores s USING (store_id)
GROUP BY city
ORDER BY total_after DESC;

```

MySQL Workbench

Local instance MySQL80 x unconnected x

File Edit View Query Database Server Tools Scripting Help

Navigator

SQL File 4*

Find & Replace

Replace All Replace

```

74 SELECT city,
75     COUNT(*) AS event_count,
76     SUM('quantity_sold(before_promo)') AS total_before,
77     SUM('quantity_sold(after_promo)') AS total_after
78 FROM fact_events f
79 JOIN dim_stores s USING (store_id)
80 GROUP BY city
81 ORDER BY total_after DESC;
82
83

```

Result Grid

city	event_count	total_before	total_after
Bengaluru	300	49171	105141
Chennai	240	39505	83273
Hyderabad	210	34363	69399
Coimbatore	150	18150	38900
Mysuru	120	18569	37470
Vaishampatnam	150	17175	33916
Madurai	120	14458	31169
Mangalore	90	7529	14929
Vijayawada	60	5297	11106
Tiruvandrum	60	4811	10170

Connection Details

Name: Local instance M
Host: localhost
Port: 3306
Login User: root
Current User: root@localhost
SSL cipher: TLS_AES_128_GCM_SHA256

Server

Product: MySQL Commur
Version: 8.0.46

Connector

Version: C++ 9.6.0

Object Info

Query Completed

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

-- Q10

```

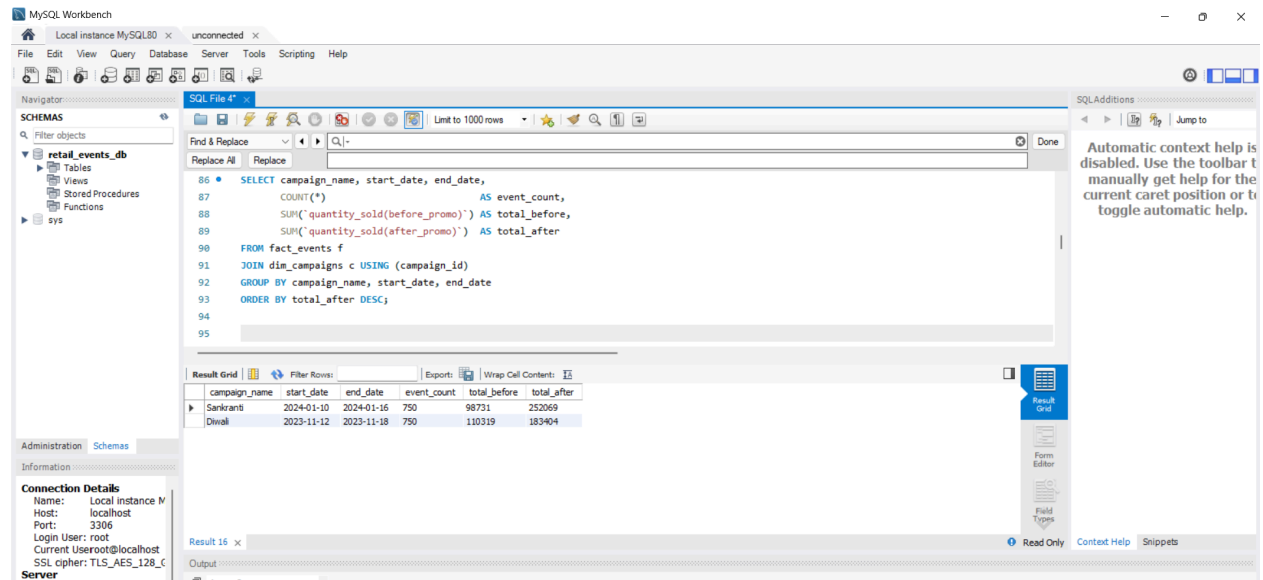
SELECT campaign_name, start_date, end_date,

```

```

COUNT(*)                AS event_count,
SUM(`quantity_sold(before_promo)`) AS total_before,
SUM(`quantity_sold(after_promo)`) AS total_after
FROM fact_events f
JOIN dim_campaigns c USING (campaign_id)
GROUP BY campaign_name, start_date, end_date
ORDER BY total_after DESC;

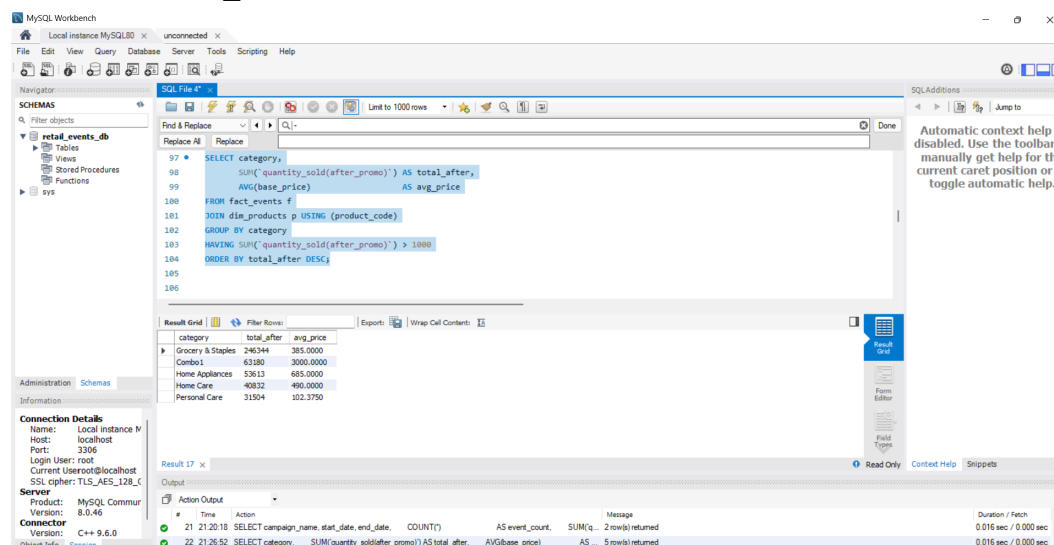
```



```

-- Q11
SELECT category,
       SUM(`quantity_sold(after_promo)`) AS total_after,
       AVG(base_price)                AS avg_price
FROM fact_events f
JOIN dim_products p USING (product_code)
GROUP BY category
HAVING SUM(`quantity_sold(after_promo)`) > 1000
ORDER BY total_after DESC;

```



```

-- Q12
SELECT city, category,

```

```

SUM(`quantity_sold(after_promo)`) AS total_after
FROM fact_events f
JOIN dim_stores s USING (store_id)
JOIN dim_products p USING (product_code)
GROUP BY city, category
ORDER BY city, total_after DESC;

```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```

SELECT city, category,
SUM(`quantity_sold(after_promo)`) AS total_after
FROM fact_events f
JOIN dim_stores s USING (store_id)
JOIN dim_products p USING (product_code)
GROUP BY city, category
ORDER BY city, total_after DESC;

```

The Results Grid shows the following data:

category	total_after	avg_price
Grocery & Staples	246344	385.0000
Combo1	63180	3000.0000
Home Appliances	53613	685.0000
Home Care	40832	490.0000
Personal Care	31504	102.3750

The Connection Details panel on the left shows the following information:

- Name: Local instance M
- Host: localhost
- Port: 3306
- Login User: root
- Current User: root@localhost
- SSL cipher: TLS_AES_128_GCM_SHA256
- Product: MySQL Commu
- Version: 8.0.46
- Connector: C++ 9.6.0

The Output panel at the bottom shows the execution log:

#	Time	Action	Message	Duration / Fetch
21	21:20:18	SELECT campaign_name, start_date, end_date, COUNT(*) AS event_count, SUM(q...	2 row(s) returned	0.016 sec / 0.000 sec
22	21:26:52	SELECT category, SUM(quantity_sold(after_promo)) AS total_after, AVG(base_price) AS ...	5 row(s) returned	0.016 sec / 0.000 sec

-- Q13

```

SELECT product_name, category,
SUM(`quantity_sold(before_promo)`) AS total_before,
SUM(`quantity_sold(after_promo)`) AS total_after,
SUM(`quantity_sold(after_promo)`) - SUM(`quantity_sold(before_promo)`) AS
quantity_change,
ROUND((SUM(`quantity_sold(after_promo)`) - SUM(`quantity_sold(before_promo)`)
/ NULLIF(SUM(`quantity_sold(before_promo)`), 0) * 100, 2) AS pct_change
FROM fact_events f
JOIN dim_products p USING (product_code)
GROUP BY product_code, product_name, category
ORDER BY pct_change DESC;

```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

119 SELECT product_name, category,
120        SUM('quantity_sold(before_promo)') AS total_before,
121        SUM('quantity_sold(after_promo)') AS total_after,
122        SUM('quantity_sold(after_promo)') - SUM('quantity_sold(before_promo)') AS quantity_change,
123        ROUND((SUM('quantity_sold(after_promo)') - SUM('quantity_sold(before_promo)'))
124              / NULLIF(SUM('quantity_sold(before_promo)'), 0) * 100, 2) AS pct_change
125 FROM fact_events f
126 JOIN dim_products p USING (product_code)
127 GROUP BY product_code, product_name, category
128 ORDER BY pct_change DESC;

```

The Results Grid shows the following data:

product_name	category	total_before	total_after	quantity_change	pct_change
Atiq_waterproof_Immersion_Rod	Home Appliances	6468	23685	17217	266.19
Atiq_High_Glo_15W_LED_Bulb	Home Appliances	8245	29928	21683	262.98
Atiq_Double_Bedsheet_set	Home Care	4203	19058	10855	256.27
Atiq_Curtains	Home Care	4992	16317	11725	235.34
Atiq_Home_Essential_8_Product_Combo	Combo1	22299	63180	40881	183.33
Atiq_Farm_Chakki_Atta (1KG)	Grocery & Staples	32340	81290	48950	151.36
Atiq_Sunflower_Oil (1L)	Grocery & Staples	31309	74478	43169	137.88
Atiq_Masoor_Dal (1KG)	Grocery & Staples	26040	37341	11301	43.40
Atiq_Sonamasun_Rice (10KG)	Grocery & Staples	37281	53235	15954	42.79
Atiq_Ready_Kitchen_Rotiv (700MM)	Personal Care	5267	7037	1765	33.57

The Output pane shows the execution results:

```

# Time Action Message Duration / Fetch
23 21:26:52 SELECT category, SUM(quantity_sold(after_promo)) AS total_after, AVG(base_price) AS ... 5 row(s) returned 0.015 sec / 0.000 sec
23 21:29:12 SELECT product_name, category, SUM(quantity_sold(before_promo)) AS total_before, SUM(quantit... 15 row(s) returned 0.015 sec / 0.000 sec

```

-- Q14

```

SELECT campaign_name, promo_type,
       COUNT(*) AS event_count,
       SUM('quantity_sold(before_promo)') AS total_before,
       SUM('quantity_sold(after_promo)') AS total_after,
       SUM('quantity_sold(after_promo)') - SUM('quantity_sold(before_promo)') AS
quantity_change
FROM fact_events f
JOIN dim_campaigns c USING (campaign_id)
GROUP BY campaign_name, promo_type
ORDER BY campaign_name, quantity_change DESC;

```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

131 SELECT campaign_name, promo_type,
132        COUNT(*) AS event_count,
133        SUM('quantity_sold(before_promo)') AS total_before,
134        SUM('quantity_sold(after_promo)') AS total_after,
135        SUM('quantity_sold(after_promo)') - SUM('quantity_sold(before_promo)') AS quantity_change
136 FROM fact_events f
137 JOIN dim_campaigns c USING (campaign_id)
138 GROUP BY campaign_name, promo_type
139 ORDER BY campaign_name, quantity_change DESC;
140

```

The Results Grid shows the following data:

campaign_name	promo_type	event_count	total_before	total_after	quantity_change
Divall	500 Cashback	50	16791	50769	33978
Divall	BOGOF	200	10024	34461	24437
Divall	33% OFF	100	29152	40117	13965
Divall	50% OFF	200	16843	22074	5231
Divall	25% OFF	200	37509	32983	-4526
Sankranti	BOGOF	300	48156	180792	132636
Sankranti	33% OFF	100	34169	47459	13290
Sankranti	500 Cashback	50	5508	12411	6903
Sankranti	50% OFF	100	4400	6100	1700
Sankranti	25% OFF	200	6468	9307	-1901

The Output pane shows the execution results:

```

# Time Action Message Duration / Fetch
23 21:29:12 SELECT product_name, category, SUM(quantity_sold(before_promo)) AS total_before, SUM(quantit... 15 row(s) returned 0.015 sec / 0.000 sec
24 21:29:44 SELECT campaign_name, promo_type, COUNT(*) AS event_count, SUM(quantity_s... 10 row(s) returned 0.000 sec / 0.000 sec

```

-- Q15

```

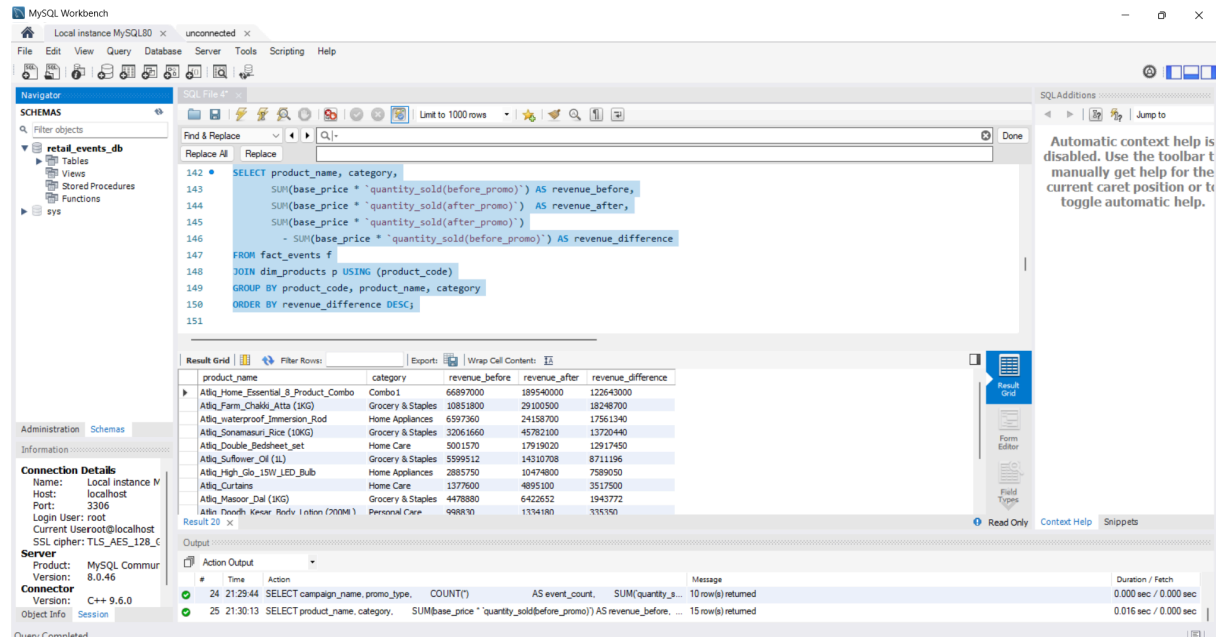
SELECT product_name, category,

```

```

SUM(base_price * `quantity_sold(before_promo)`) AS revenue_before,
SUM(base_price * `quantity_sold(after_promo)`) AS revenue_after,
SUM(base_price * `quantity_sold(after_promo)`)
- SUM(base_price * `quantity_sold(before_promo)`) AS revenue_difference
FROM fact_events f
JOIN dim_products p USING (product_code)
GROUP BY product_code, product_name, category
ORDER BY revenue_difference DESC;

```



```

-- Q16
WITH t AS (
    SELECT promo_type,
           SUM(`quantity_sold(before_promo)`) AS total_before,
           SUM(`quantity_sold(after_promo)`) AS total_after
    FROM fact_events
    GROUP BY promo_type
), pct AS (
    SELECT *, ROUND((total_after - total_before) / NULLIF(total_before, 0) * 100, 2) AS
percentage_change
    FROM t
)
SELECT *,
       CASE WHEN percentage_change >= 50 THEN 'High Impact'
            WHEN percentage_change >= 20 THEN 'Medium Impact'
            ELSE 'Low Impact' END AS performance_category
FROM pct
ORDER BY percentage_change DESC;

```

MySQL Workbench

Local instance MySQL80 x unconnected x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

retail_events_db

Tables

Views

Stored Procedures

Functions

sys

Administration Schemas

Information

Connection Details

Name: Local instance M

Host: localhost

Port: 3306

Login User: root

Current User: root@localhost

SSL cipher: TLS_AES_128_GCM_SHA256

Server

Product: MySQL Commu

Version: 8.0.46

Connector

Version: C++ 9.6.0

Object Info Session

SQL File 47

Find & Replace

Replace All Replace

160 SELECT *, ROUND((total_after - total_before) / NULLIF(total_before, 0) * 100, 2) AS percentage_change

161 FROM t

162)

163 SELECT *,

164 CASE WHEN percentage_change >= 50 THEN 'High Impact'

165 WHEN percentage_change >= 20 THEN 'Medium Impact'

166 ELSE 'Low Impact' END AS performance_category

167 FROM pct

168 ORDER BY percentage_change DESC;

169

Result Grid

promo_type	total_before	total_after	percentage_change	performance_category
BOGOF	58180	215253	269.98	High Impact
500 Cashback	22299	63180	183.33	High Impact
33% OFF	63321	90576	43.04	Medium Impact
50% OFF	21243	28174	33.63	Medium Impact
25% OFF	44007	38290	-12.99	Low Impact

Result 21 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
25	21:30:13	SELECT product_name, category, SUM(base_price * quantity_sold(before_promo)) AS revenue_before, ...	15 row(s) returned	0.016 sec / 0.000 sec
26	21:31:02	WITH t AS (SELECT promo_type, SUM(quantity_sold(before_promo)) AS total_before, SUM(qu...	5 row(s) returned	0.015 sec / 0.000 sec

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

-- Q17

WITH t AS (

SELECT category, product_name,

SUM('quantity_sold(after_promo)') AS total_quantity_after

FROM fact_events f

JOIN dim_products p USING (product_code)

GROUP BY category, product_code, product_name

), r AS (

SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY total_quantity_after DESC) AS category_rank

FROM t

)

SELECT * FROM r WHERE category_rank <= 2;

MySQL Workbench

Local instance MySQL80 x unconnected x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

retail_events_db

Tables

Views

Stored Procedures

Functions

sys

Administration Schemas

Information

Connection Details

Name: Local instance M

Host: localhost

Port: 3306

Login User: root

Current User: root@localhost

SSL cipher: TLS_AES_128_GCM_SHA256

Server

Product: MySQL Commu

Version: 8.0.46

Connector

Version: C++ 9.6.0

Object Info Session

SQL File 47

Find & Replace

Replace All Replace

179 FROM t

180)

181 SELECT * FROM r WHERE category_rank <= 2;

182

183 -- Q18

184 WITH t AS (

185 SELECT city, store_id,

186 SUM('quantity_sold(after_promo)') AS total_quantity_after

187 FROM fact_events f

188 JOIN dim_stores s USING (store_id)

Result Grid

city	store_id	total_quantity_after	city_rank
Bengaluru	STBLR-7	11866	1
Bengaluru	STBLR-6	11601	2
Chennai	STCHE-4	11547	1
Chennai	STCHE-7	11546	2
Coimbatore	STCR-0	8657	1
Coimbatore	STCR-2	8535	2
Hyderabad	STHYD-2	10822	1
Hyderabad	STHYD-0	10522	2
Madurai	STMDU-0	8310	1
Madurai	STMUR-7	7644	2

Result 23 x

Output

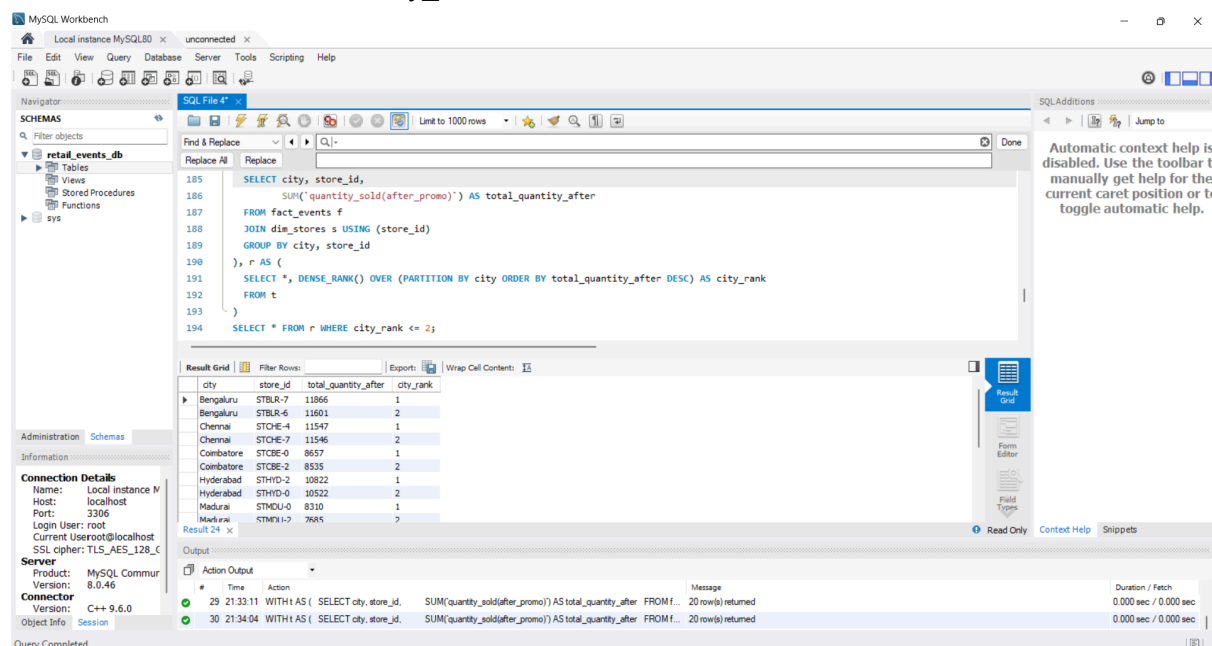
Action Output

#	Time	Action	Message	Duration / Fetch
28	21:31:40	WITH t AS (SELECT category, product_name, SUM(quantity_sold(after_promo)) AS total_quantity_ah...	9 row(s) returned	0.016 sec / 0.000 sec
29	21:33:11	WITH t AS (SELECT city, store_id, SUM(quantity_sold(after_promo)) AS total_quantity_after FROM t...	20 row(s) returned	0.000 sec / 0.000 sec

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

```
-- Q18
WITH t AS (
    SELECT city, store_id,
           SUM(`quantity_sold(after_promo)`) AS total_quantity_after
    FROM fact_events f
    JOIN dim_stores s USING (store_id)
    GROUP BY city, store_id
), r AS (
    SELECT *, DENSE_RANK() OVER (PARTITION BY city ORDER BY total_quantity_after
DESC) AS city_rank
    FROM t
)
SELECT * FROM r WHERE city_rank <= 2;
```



```
-- Q19
WITH t AS (
    SELECT campaign_name, product_name,
           SUM(`quantity_sold(before_promo)`) AS total_before,
           SUM(`quantity_sold(after_promo)`) AS total_after,
           SUM(`quantity_sold(after_promo)`) - SUM(`quantity_sold(before_promo)`) AS
quantity_change,
           ROUND((SUM(`quantity_sold(after_promo)`) - SUM(`quantity_sold(before_promo)`)
    / NULLIF(SUM(`quantity_sold(before_promo)`), 0) * 100, 2) AS
percentage_change
    FROM fact_events f
    JOIN dim_campaigns c USING (campaign_id)
    JOIN dim_products p USING (product_code)
    GROUP BY campaign_name, product_code, product_name
), r AS (
    SELECT *, DENSE_RANK() OVER (PARTITION BY campaign_name ORDER BY
percentage_change DESC) AS campaign_rank
```

FROM t

)

SELECT * FROM r WHERE campaign_rank <= 3;

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
191 SELECT *, DENSE_RANK() OVER (PARTITION BY city ORDER BY total_quantity_after DESC) AS city_rank
192 FROM t
193 )
194 SELECT * FROM r WHERE city_rank <= 3;
195
196 -- Q19
197 WITH t AS (
198     SELECT campaign_name, product_name,
199           SUM("quantity_sold(before_promo)") AS total_before,
200           SUM("quantity_sold(after_promo)") AS total_after,
```

The Results Grid shows the following data:

campaign_name	product_name	total_before	total_after	quantity_change	percentage_change	campaign_rank
Divall	Atiq_waterproof_Immersion_Rod	2015	6950	4935	244.91	1
Divall	Atiq_Curtains	2680	9214	6534	243.81	2
Divall	Atiq_High_Glo_15W_LED_Bulb	3215	11053	7838	243.79	3
Sankranti	Atiq_Suflower_Oil (L)	16257	61185	44928	276.36	1
Sankranti	Atiq_waterproof_Immersion_Rod	4453	16735	12282	276.81	2
Sankranti	Atiq_High_Glo_15W_LED_Bulb	5030	18875	13845	275.25	3

The Output tab shows the execution of the query with the following messages:

```
30 21:34:04 WITH t AS ( SELECT city, store_id, SUM(quantity_sold(after_promo)) AS total_quantity_after FROM t... 20 row(s) returned
31 21:39:32 WITH t AS ( SELECT campaign_name, product_name, SUM(quantity_sold(before_promo)) AS total_b... 6 row(s) returned
```

-- Q20

WITH t AS (

SELECT product_name, category,

COUNT(*) AS event_count,

SUM("quantity_sold(before_promo)") AS total_before,

SUM("quantity_sold(after_promo)") AS total_after,

SUM("quantity_sold(after_promo)") - SUM("quantity_sold(before_promo)") AS

quantity_change,

ROUND((SUM("quantity_sold(after_promo)") - SUM("quantity_sold(before_promo)"))
/ NULLIF(SUM("quantity_sold(before_promo)"), 0) * 100, 2) AS

percentage_change,

SUM(base_price * "quantity_sold(before_promo)") AS revenue_before,

SUM(base_price * "quantity_sold(after_promo)") AS revenue_after,

SUM(base_price * "quantity_sold(after_promo)")

- SUM(base_price * "quantity_sold(before_promo)") AS revenue_change,

ROUND(AVG(base_price), 2) AS avg_base_price

FROM fact_events f

JOIN dim_products p USING (product_code)

GROUP BY product_code, product_name, category

), r AS (

SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue_change
DESC) AS category_rank

FROM t

)

SELECT * FROM r WHERE category_rank <= 2

ORDER BY category, category_rank;

MySQL Workbench

Local instance MySQL80 x unconnected x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

retail_events_db

Tables

Views

Stored Procedures

Functions

sys

Administration Schemas

Information

Connection Details

Name: Local instance M

Host: localhost

Port: 3306

Login User: root

Current User: root@localhost

SSL cipher: TLS_AES_128_GCM_SHA256

Server

Product: MySQL Commu

Version: 8.0.46

Connector

Version: C++ 9.6.0

Object Info Session

Query Completed

SQL File #1

Find & Replace

Replace All

Replace

Limit to 1000 rows

228 FROM fact_events f

229 JOIN dim_products p USING (product_code)

230 GROUP BY product_code, product_name, category

231), r AS (

232 SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue_change DESC) AS category_rank

233 FROM t

234)

235 SELECT * FROM r WHERE category_rank <= 2

236 ORDER BY category, category_rank;

237

Result Grid

Filter Rows

Export

Wrap Cell Contents

product_name	category	event_count	total_before	total_after	quantity_change	percentage_change	revenue_before	revenue_after	revenue_s
Atliq_Home_Essential_8_Product_Combo	Combo 1	100	22299	63180	40881	183.33	66897000	189540000	122643000
Atliq_Farm_Chalks_Atta (1KG)	Grocery & Staples	100	32340	81290	48950	151.36	10851800	29100500	18248700
Atliq_Sonamasun_Rice (10KG)	Grocery & Staples	100	37281	53235	15954	42.79	32061660	45782100	13720440
Atliq_waterproof_Immersion_Rod	Home Appliances	100	6468	23685	17217	266.19	6597360	24158700	17561340
Atliq_High_Glo_15W_LED_Bulb	Home Appliances	100	8245	29928	21683	262.98	2885750	10474800	7589050
Atliq_Double_Bedsheet_set	Home Care	100	4203	15058	10855	258.27	5001570	17919020	12917450
Atliq_Curtains	Home Care	100	4992	16317	11725	235.34	1377600	4895100	3517500
Atliq_Doodh_Kesar_Body_Lotion (200ML)	Personal Care	100	5257	7022	1765	33.57	998830	1334180	335350
Atlin 1 line Cool Refreshing Face (125GM)	Personal Care	100	7718	10780	2962	38.39	4785116	6177900	1588444

Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
31	21:39:32	WITH t AS (SELECT campaign_name, product_name, SUM(quantity_sold(before_promo)) AS total_b...	6 row(s) returned	0.016 sec / 0.000 sec
32	21:39:55	WITH t AS (SELECT product_name, category, COUNT(*) AS event_count, SUM...	9 row(s) returned	0.016 sec / 0.000 sec

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.